Overestimation Bias in Value-Based RL: From Q-Learning to Double DQN

FRA503 Deep Reinforcement Learning Final Project

May 24, 2026

1 Project Overview

Value-based Reinforcement Learning (RL) relies heavily on estimating the expected return of actions to derive optimal policies. The cornerstone algorithm, Q-Learning, utilizes the max operator to estimate the value of the next state. While mathematically elegant, this introduces a systematic overestimation bias because the maximum of noisy estimates is consistently greater than the true maximum expected value. Over time, this positive bias can compound, leading to suboptimal policies. This project aims to empirically observe, measure, and analyze this overestimation bias, transitioning from simple Tabular RL to complex Deep RL using neural network function approximation.

2 Scope and Objectives

2.1 Scope Revision from Proposal

The original project proposal was titled "Overestimation Bias in Value-Based RL: From Q-Learning to Double DQN with Prioritized Experience Replay". The initial scope included the implementation of Prioritized Experience Replay (PER). However, during the implementation of the Deep RL phase, a more fundamental and severe stability problem emerged: the *Deadly Triad*. Function approximation combined with bootstrapping and off-policy learning caused the Q-values to diverge massively.

Consequently, the final project scope was revised. The implementation of PER was moved to Future Work. The revised scope focuses heavily on hyperparameter tuning to mitigate the Deadly Triad, enabling a stable environment to accurately measure the underlying overestimation bias in DQN and Double DQN.

2.2 Final Scope

The experiment is divided into two phases:

- **Phase 1 (Tabular RL):** Analyzing Q-Learning vs. Double Q-Learning to establish a baseline for overestimation bias.
- **Phase 2 (Deep RL):** Analyzing DQN vs. Double DQN, focusing on the Deadly Triad instability and the subsequent hyperparameter tuning required to stabilize training.

2.3 Evaluation Metrics

- **Primary Metric:** *MaxQ Bias*. This is the core metric as it directly measures the effect of the max operator against true empirical returns.

- **Secondary Metrics:** Average Reward (policy performance), Episode Duration (stability), Action Distribution (aggressiveness), and Pole Angle Trajectory (smoothness).

3 Problem Formulation

The CartPole stabilization task is modeled as a Markov Decision Process (MDP) defined by a tuple $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$. The observation state consists of four continuous variables: cart position, pole angle, cart velocity, and pole angular velocity. The action space $\mathcal{A}$ is discretized into five force levels: $[-1.0, -0.5, 0.0, 0.5, 1.0]$. The discount factor is $\gamma = 0.99$ (tuned to 0.95 in Deep RL).

The optimal action-value function $Q^*(s, a)$ is estimated using the standard Q-Learning update rule:

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s,a) \right] \tag{1}$$

The root cause of overestimation lies in the target $r + \gamma \max_{a'} Q(s', a')$. Let X be the noisy estimate of the true value. Due to Jensen's inequality and the convexity of the max function, we have:

$$\mathbb{E}[\max(X)] \geq \max(\mathbb{E}[X]) \tag{2}$$

Using the same noisy Q-table to both *select* the best action and *evaluate* its value guarantees a positive bias. We define our primary metric as:

$$\text{MaxQ Bias}(s_t) = \max_a Q(s_t, a) - G_t^{MC} \tag{3}$$

where G_t^{MC} is the actual Monte Carlo return achieved by following a greedy policy from state s_t. Positive values indicate overestimation, while negative values indicate underestimation.

4 Techniques

4.1 Algorithms

1. **Q-Learning:** Standard tabular implementation using a discretized observation space.
2. **Double Q-Learning:** Decouples selection and evaluation using two separate tables (Q_A and Q_B) to eliminate positive bias.
3. **DQN:** Employs a 4-layer feedforward neural network, an experience replay buffer, and a slowly updating target network.
4. **Double DQN:** Uses the online network to select the action $a^* = \arg\max_a Q_{online}(s', a)$, and the target network to evaluate it $Q_{target}(s', a^*)$.

4.2 The Deadly Triad and Hyperparameter Tuning

The *Deadly Triad* occurs when three elements are combined: Function Approximation (Neural Networks), Bootstrapping (updating estimates based on other estimates), and Off-policy learning (learning from an experience replay). In our initial Deep RL experiments, this caused the Q-values to diverge into extreme underestimation. To stabilize the training, we tuned the hyperparameters as shown in Table **??**.

Hyperparameter	Before	After	Reason for Stability Improvement
`target_update`	1000	100	More frequent updates reduce the stale target problem.
`gamma` (γ)	0.99	0.95	Reduces long-horizon bootstrapping pressure.
`gradient_clip`	10.0	1.0	Prevents gradient explosion from destabilizing weights.
`min_buffer`	1000	5000	Improves replay diversity before initial learning begins.

Table 1: Hyperparameter tuning applied to mitigate Deadly Triad divergence.

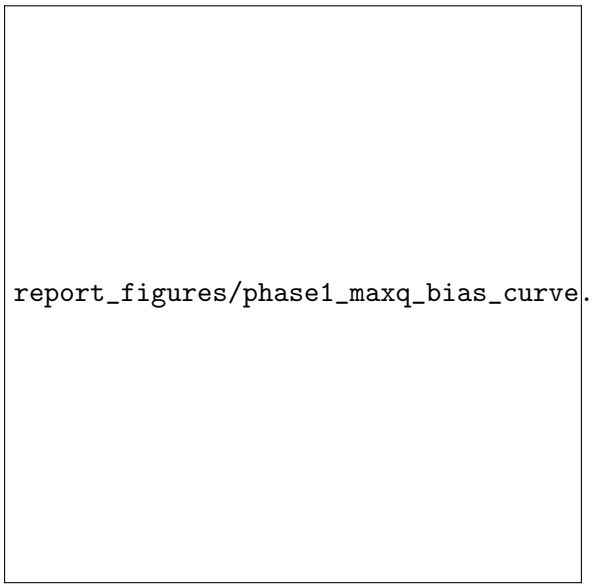
5 Simulation Setup

All experiments were conducted on the `Stabilize-Isaac-Cartpole-v0` environment in IsaacLab. For Phase 1, the continuous observation space was discretized into bins $[8, 16, 8, 8]$. The experiments were run across 5 random seeds to ensure statistical significance. During training, a greedy evaluation phase was triggered periodically (e.g., every 100 episodes) where exploration (ϵ) was disabled. The agent played out the episode, and the true Monte Carlo returns were collected and compared against the agent's internal Q-estimates to compute the MaxQ Bias.

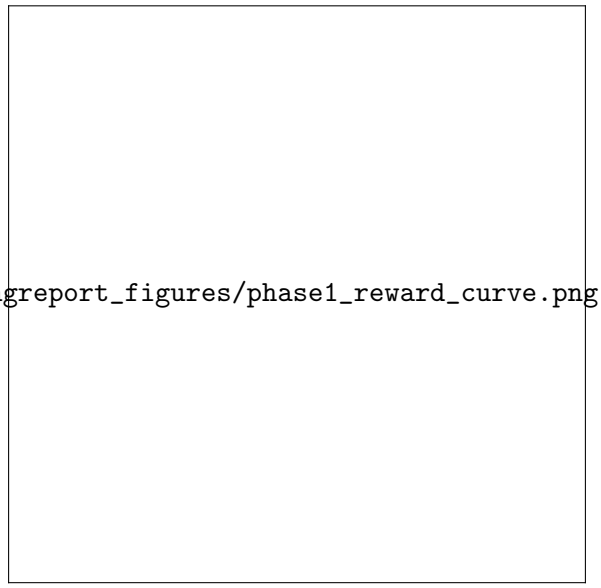
6 Results

6.1 Phase 1: Tabular RL Results

Figure **??** illustrates the primary finding of Phase 1: Q-Learning explicitly suffers from positive overestimation bias. The MaxQ Bias for Q-Learning climbs into the positive domain, confirming theoretical predictions. Conversely, Double Q-Learning effectively suppresses this, remaining consistently negative (conservative underestimation).



(a) MaxQ Bias Curve

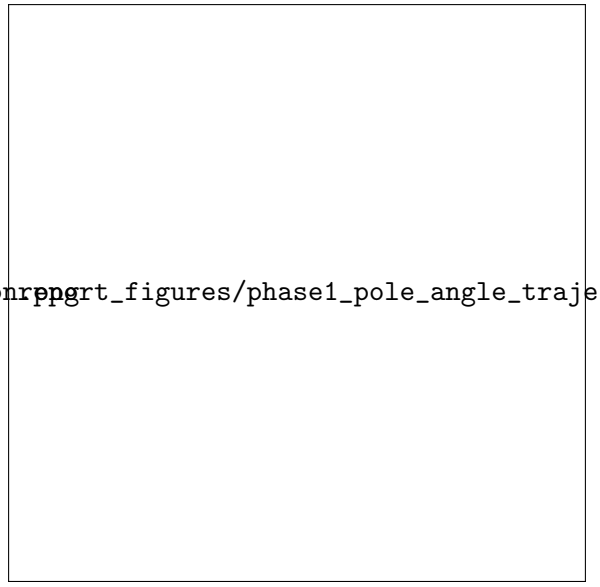


(b) Average Reward

Figure 1: Phase 1 Comparison between Q-Learning and Double Q-Learning.

Despite the positive bias, Q-Learning often resulted in slightly different control strategies. As seen in Figure **??**, both algorithms maintained the pole angle, but the action distributions varied.

(a) Action Distribution



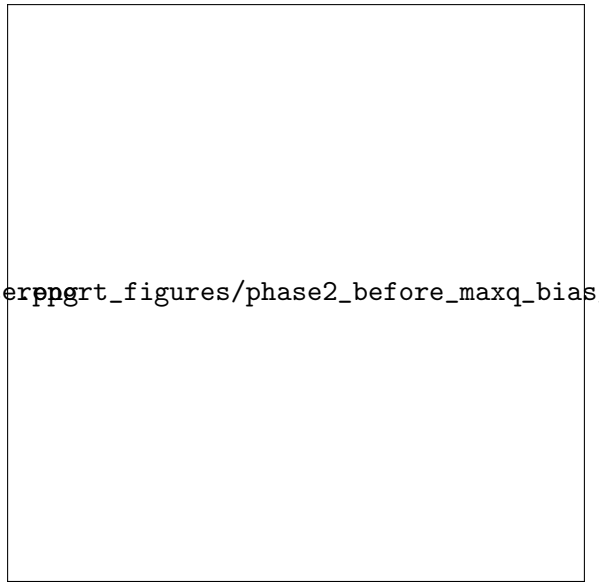
(b) Pole Angle Trajectory

Figure 2: Phase 1 Behavioral Analysis.

6.2 Phase 2: Deep RL before Hyperparameter Tuning

Before tuning, the introduction of Neural Networks triggered the Deadly Triad. Instead of standard overestimation, the networks failed to anchor to true returns, resulting in severe divergence. Figure **??** shows Q-values plummeting to approximately $-17,000$, despite the maximum possible Monte Carlo return being around $+100$.

(a) Q-Value Divergence



(b) MaxQ Bias (Extreme Underestimation)

Figure 3: Phase 2 (Before Tuning): The Deadly Triad causing severe instability.

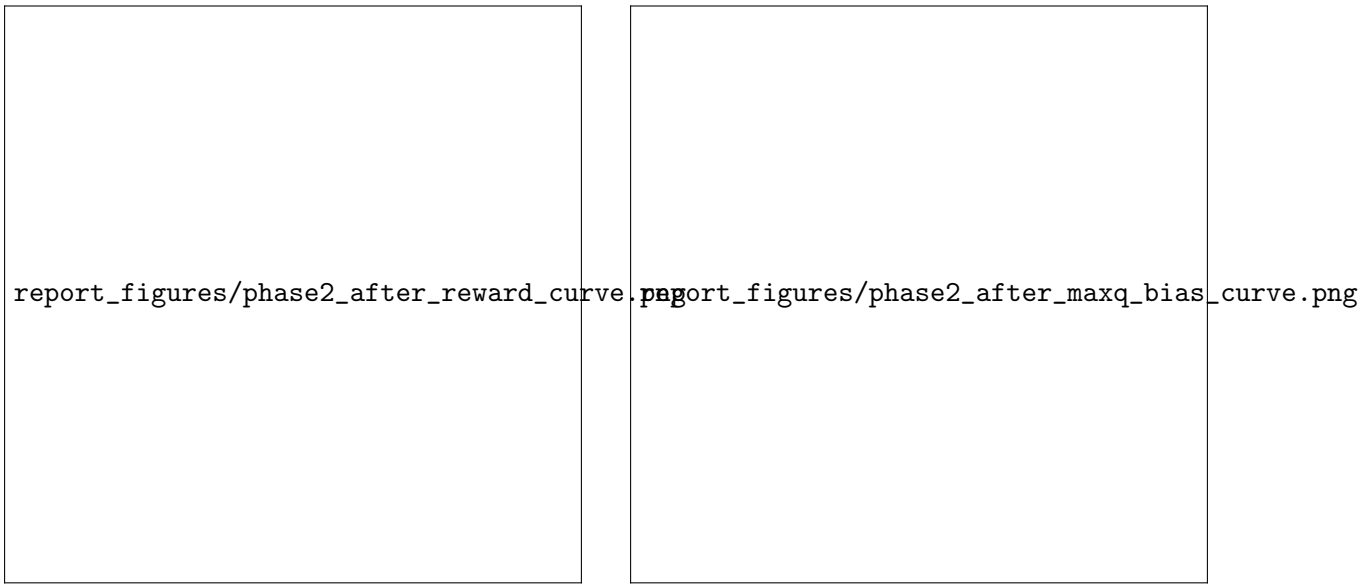
6.3 Phase 2: Deep RL after Hyperparameter Tuning

After applying the hyperparameter adjustments (notably reducing γ to 0.95 and accelerating the target network update interval), the Deadly Triad was successfully neutralized for both

algorithms. As shown in Figure **??**, both DQN and Double DQN achieved stable training without any catastrophic divergence.

Most importantly, the true underlying bias behavior emerged. The *MaxQ Bias* for standard DQN stabilized at +0.297, indicating a distinct positive overestimation bias. In contrast, Double DQN successfully suppressed this inflation, achieving a lower bias of +0.202. While the neural network function approximation compresses the gap compared to the tabular setting, the core finding holds true: decoupling selection from evaluation in Double DQN actively reduces overestimation.

Interestingly, similar to the tabular phase, the slightly optimistic DQN achieved a higher final average reward (355.7) compared to the more conservative Double DQN (285.4).



(a) Average Reward after Hyperparameter Tuning (b) MaxQ Bias after Hyperparameter Tuning

Figure 4: Phase 2 (After Tuning): Stable learning showing DQN's overestimation (+0.297) vs Double DQN's reduced bias (+0.202).

7 Analysis

1. **Does overestimation bias exist in Q-Learning?** Yes. The Tabular Phase 1 results definitively showed the MaxQ Bias crossing into positive territory, proving that the max operator inflates value estimates.

2. **Does Double Q-Learning reduce positive overestimation?** Yes. By decoupling action selection from evaluation, Double Q-Learning kept the MaxQ Bias in the negative domain throughout training, completely suppressing positive overestimation.

3. **Does Double DQN reduce the max-operator effect compared to DQN?** Yes. Once the Deadly Triad was neutralized, Double DQN empirically demonstrated a lower MaxQ Bias (+0.202) compared to standard DQN (+0.297). The gap is narrower than in Tabular RL due to neural network generalization, but the bias reduction mechanic is clearly functional.

4. **How did the Deadly Triad affect Deep RL training?** It masked standard overestimation bias entirely by causing catastrophic underestimation. The compounding errors from bootstrapping off-policy neural network predictions forced Q-values to diverge into the negative tens-of-thousands.

5. **Did hyperparameter tuning reduce divergence or improve stability?** Yes. By lowering γ to 0.95, shrinking the target update interval to 100, and clipping gradients to 1.0, the Q-values were anchored to realistic bounds (around 0 to 20), eliminating the divergence.

6. **Does lower MaxQ Bias always mean higher reward?** No. Bias reduction and policy performance are related but not identical. While Double Q-Learning effectively removed bias, standard Q-Learning (and occasionally standard DQN) often learned stabilizing policies faster because optimistic/aggressive value estimates encourage faster exploration of critical states.

8 Conclusion and Future Work

This project successfully demonstrated the existence of overestimation bias in Q-Learning and its resolution via Double Q-Learning in a tabular environment. Transitioning to Deep RL highlighted the complexity of function approximation, where the Deadly Triad initially caused severe training instability. Extensive hyperparameter tuning was required to stabilize the networks before meaningful bias analysis could be conducted. The results confirm that while bias reduction algorithms work as theorized, policy performance relies heavily on a delicate balance of stability, exploration, and accurate value anchoring.

Future Work: The original proposal's scope included Prioritized Experience Replay (PER). Implementing PER alongside the stabilized Double DQN architecture is the immediate next step. Additionally, longer training regimens across more random seeds will help validate the statistical significance of the deep RL findings.